UniSphere: An Integrated College Club Management and Community Engagement Portal

M. SRUJANYA
UG Student, Dept of CSE
CMR Technical Campus
Hyderabad, Telangana, India- 501401
237r1a05ba@cmrtc.ac.in

DR. V NARESH KUMAR
Associate Professor, Dept of CSE
CMR Technical Campus
Hyderabad, Telangana, India- 501401
nareshkumar.cse@cmrtc.ac.in

E. SAKETH
UG Student, Dept of CSE
CMR Technical Campus
Hyderabad, Telangana, India- 501401
237r1a05aj@cmrtc.ac.in

J. MAHALAXMI
UG Student, Dept of CSE
CMR Technical Campus
Hyderabad, Telangana, India- 501401
237r1a05an@cmrtc.ac.in

P. SANTHUJA
Assistant Professor, Dept of CSE
CMR Technical Campus
Hyderabad, Telangana, India- 501401
Santhuja.pandu@gmail.com

MAM
Assistant Professor, Dept of CSE
CMR Technical Campus
Hyderabad, Telangana, India- 501401
saraswatiparchaki@gmail.com

***Abstract*— Student clubs on most college campuses are still coordinated through scattered WhatsApp groups, paper or spreadsheet-based membership forms, and manually organised event sign-ups, which makes it hard for students to discover clubs, for coordinators to track membership, and for event photos and announcements to reach the people who would want them. This paper presents UniSphere, a web-based portal that brings club discovery, membership requests, event registration, announcements, and event photo galleries into a single system built around a five-tier role hierarchy — Visitor, Student, Club Core Lead, Faculty Coordinator, and Admin — with each tier building on the permissions of the one below it. Visitors can browse clubs and public event photos; Students can sign up, apply to clubs, register for events, and track their own membership status; Club Core Leads review membership requests and manage events and media for their own club; Faculty Coordinators have oversight across clubs; and Admins manage the platform as a whole. The system was built using React on the front end and an Express-based REST API on the back end, with JSON Web Tokens used to authenticate users and enforce these role boundaries. Manual testing of the core workflows — signup, membership requests, event registration, and photo uploads — showed that each role saw only the dashboard and actions appropriate to it, suggesting UniSphere is a workable alternative to running college clubs through disconnected manual tools.**

Keywords: role-based access control, JWT authentication, club management system, event registration, photo gallery, web application, REST API.

I. INTRODUCTION

A. Motivation and Context

Most college campuses, including CMR Technical Campus (CMRTC), run a number of student clubs covering different interests — cultural activities, coding and technical events, photography, debate, NCC, and social service, to name a few. Being part of these clubs matters for a student's overall growth and is often noted during placement conversations and institutional accreditation reviews. Despite this, very few colleges have built proper software for managing them; clubs are usually left to run on whatever combination of chat apps and spreadsheets a particular batch of coordinators happens to prefer.

B. Problem Statement

Looking at how clubs are actually run on most campuses today, a few recurring problems stand out, and these are what UniSphere is built to address:

- Club information and updates are usually posted inside temporary WhatsApp or Telegram groups, so students who are not already in the group miss announcements, and older messages are difficult to find again later.
- Membership is tracked through separate Google Forms or paper registers maintained by each club on its own, which makes it easy for the same student to be counted twice and hard for a coordinator to confirm who is actually an active member.
- Event sign-ups are handled manually, with no shared way to track how many seats are left or to confirm who has actually registered, which leads to confusion at the door and no clear record afterwards.
- Photos from club events end up scattered across personal phones, shared drive links, or social media posts, so there is no single place a student can go back to and find pictures from a specific event.

UniSphere was built to bring these pieces — club discovery, membership requests, event registration, announcements, and event photo galleries — into one system, with clear rules about who can view or manage what.

II. LITERATURE REVIEW

Role-based access control (RBAC) is a well-studied way of deciding who can do what inside a multi-user system. Sandhu et al. laid out the original RBAC models, separating the ideas of users, roles, and permissions [1], and this basic idea — assign permissions to roles, then assign users to roles — is what UniSphere's five-tier structure is built on. Ferraiolo et al. later formalized this into a NIST standard covering flat, hierarchical, and constrained RBAC [2]; UniSphere follows the hierarchical version, where a higher role automatically gets everything a lower role can do, plus a few extra permissions of its own.

For authentication, UniSphere uses JSON Web Tokens (JWT), as defined in RFC 7519 [3]. A JWT lets the server issue a signed token after login that the frontend then attaches to every request, so the server does not need to keep track of session state for each logged-in user separately — a convenient fit for a frontend and backend that are developed and deployed as two separate applications. This client-server split follows the REST style described by Fielding [4], where the backend exposes stateless HTTP endpoints and the frontend is responsible for the interface and for holding onto the user's session token.

For event check-in specifically, a number of existing event-management systems rely on QR or barcode-based ticket scanning, following standards such as ISO/IEC 18004 for QR symbology [5], since a phone camera is enough to confirm a ticket without any extra hardware at the door. UniSphere's current implementation manages event registration and seat tracking through its REST API rather than a scanning step, though this is a natural direction the system could be extended in, as discussed in Section VIII.

Compared to a generic college management system, which usually treats extracurricular activity as a minor add-on to academic or course modules, UniSphere is built specifically around club discovery, membership, events, and event media, rather than having these features spread thinly across a bigger, unrelated system. Section VI compares UniSphere against manual methods and generic college management approaches on specific features.

III. Methodology

A. System Architecture

UniSphere follows a decoupled architecture: a React frontend and an Express backend that communicate only through a REST API, as shown in Fig. 1.

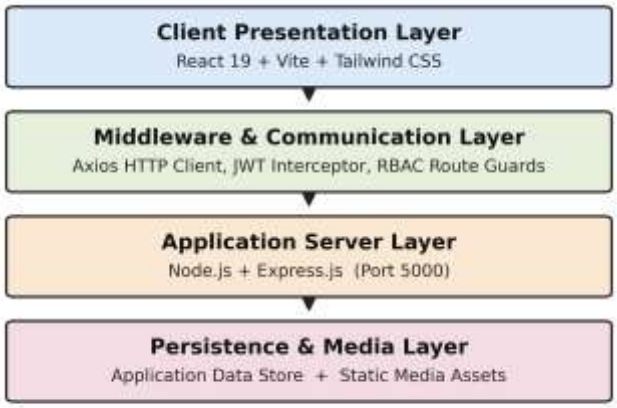

Fig. 1. Layered system architecture of UniSphere.

The frontend (built with React and Vite) contains the club-discovery pages, the login and signup flow, dashboards that change depending on the logged-in user's role, event registration screens, and the photo gallery views. An Axios-based communication layer attaches the JWT to outgoing requests and restricts navigation to pages the current role is not allowed to see. The backend, built with Express, verifies the JWT and checks the user's role before handing a request off to the relevant controller for authentication, clubs, events, requests, announcements, or media. Application data and uploaded media are kept in the backend's data store; the exact storage technology is an internal implementation detail and is not the focus of this paper.

B. Role-Based Access Control

UniSphere defines five roles, from lowest to highest access: Visitor, Student, Club Core Lead, Faculty Coordinator, and Admin. Let R = {Visitor, Student, Core, Faculty, Admin} be the set of roles and P = {BrowseClubs, ViewGallery, ApplyClub, RegisterEvent, ViewProfile, DecideRequest, PublishEvent, ManageMedia, ManageAllClubs, ManagePlatform} be the set of permissions. Each role builds on the one below it:

$$Visitor \Rightarrow \{BrowseClubs,\ ViewGallery\}$$

$$Student \Rightarrow Visitor \cup \{ApplyClub,\ RegisterEvent,\ ViewProfile\}$$

$$Core \Rightarrow Student \cup \{DecideRequestClub,\ PublishEventClub,\ ManageMediaClub\}$$

$$Faculty \Rightarrow Core \cup \{PublishEventAllClubs,\ ManageMediaAllClubs,\ OverseeAllClubs\}$$

$$Admin \Rightarrow P \quad (everything,\ including\ platform\text{-}wide\ user\ administration)$$

Fig. 2. Cumulative role hierarchy: each tier inherits every permission of the tier below it.

System Module / Operation	Visitor	Student	Club Core Lead	Faculty Coordinator	Admin
Browse clubs & public event photos	✓	✓	✓	✓	✓
View own profile & membership status	✗	✓	✓	✓	✓
Submit club membership request	✗	✓	✗	✗	✗

System Module / Operation	Visitor	Student	Club Core Lead	Faculty Coordinator	Admin
Register for events (seat booking)	✗	✓	✓	✓	✓
Download event photos	✗	✓	✓	✓	✓
Approve / reject membership requests	✗	✗	✓ (own club)	✓ (all clubs)	✓ (all clubs)
Publish & manage club events	✗	✗	✓ (own club)	✓ (all clubs)	✓ (all clubs)
Upload & manage event photo albums	✗	✗	✓ (own club)	✓ (all clubs)	✓ (all clubs)
Post club announcements	✗	✗	✓ (own club)	✓ (all clubs)	✓ (all clubs)
Platform / user administration	✗	✗	✗	✗	✓

Table I. Access matrix across the five defined roles.

C. Club and Membership Management

Students can browse the list of clubs and apply to join one through a request form. The Core Lead of that club then reviews the request and approves or rejects it; once approved, the student is added to that club's roster and can see their membership status reflected on their own profile. This replaces the earlier pattern of separate Google Forms per club, since every request goes through the same approval flow and is tied to a single student account rather than a form submission that has to be manually cross-checked.

D. Event Management and Registration

Club Core Leads and Faculty Coordinators can create and publish events for their club, and students can register for a seat directly through the platform. The system keeps track of how many seats have been booked against the event's seat limit, so coordinators have a running count of registrations instead of relying on a printed sign-up sheet.

E. Photo Gallery and Media Management

Photos are organised club-wise and, within each club, event-wise: each event can have its own album, and an album can hold many photos rather than being limited to a single image. Students and other club members can view and download the photos in an album once it has been uploaded. Club Core Leads are responsible for uploading and managing these albums for their own club's events, and Faculty Coordinators have the same ability across the clubs they oversee. This paper does not specify the exact mechanism used to host the underlying image files, since that is an internal implementation detail rather than something confirmed as part of the project description.

F. Announcements

Clubs can post announcements that appear on the platform rather than in a temporary chat group, giving students a single place to check for updates from any club rather than needing to already be part of that club's messaging group.

IV. Implementation

The frontend is built with React and communicates with an Express-based REST API. Table II illustrates the main operations the API supports in general REST style, based on the roles and workflows described in Section III; exact route names may differ in the deployed implementation.

Endpoint	Method	Access Level	Description
/auth/register	POST	Public	Register a new student account
/auth/login	POST	Public	Authenticate user; returns token and profile
/clubs	GET	Public	List all clubs (club discovery)
/requests/apply	POST	Student	Submit a club membership request
/requests/:id/status	PUT	Core / Faculty	Approve or reject a membership request
/events	POST	Core / Faculty	Create and publish a club event
/events/:id/register	POST	Student	Register for an event (seat booking)
/events/:id/gallery	POST	Core / Faculty	Upload photos to an event's album
/events/:id/gallery	GET	Student	View / download an event's photo album
/announcements	POST	Core / Faculty	Post a club announcement

Table II. Illustrative REST-style API operations.

The data model is organised around a small set of core entities — Users, Clubs, Requests, Events, Albums/Photos, and Announcements — connected through relationships that mirror how the platform is actually used: a user submits requests and registers for events, a club receives requests and hosts events, and an event can have an associated photo album.

Testing so far has been manual rather than automated: the main flows — creating an account, applying to a club, having that request approved or rejected, publishing and registering for an event, and uploading and viewing event photos — were exercised directly to check that each role's dashboard showed only the actions it should and that requests behaved as expected. No formal performance or security testing has been carried out yet, so this paper does not report response-time or load figures.

V. Result and Analysis

Working through the core flows confirmed that the role hierarchy behaves as designed: a Student account could not

reach the club-management or platform-administration screens meant for Core Leads, Faculty Coordinators, or Admins, and attempting to access those routes directly was blocked. Membership requests correctly moved from a pending state to approved or rejected once a Core Lead or Faculty Coordinator acted on them, and the student's own profile reflected the updated membership status. Event registration correctly stopped accepting new sign-ups once an event's seat limit was reached. Uploaded event photos appeared in the corresponding club's album and could be viewed and downloaded by logged-in members. At the top of the hierarchy, the Admin dashboard exposed platform-wide controls — the full club registry, user and role management, and faculty coordinator assignment — that were not reachable from any of the other three role dashboards, matching the platform-wide administration permissions defined in Table I. These observations are based on manual walkthroughs of the application rather than a formal test suite, and are reported here as qualitative evidence that the intended role separation and workflows function correctly, rather than as measured performance results.

Fig. 3–6 show representative screens from the implemented interface, illustrating the role-based dashboards and club discovery page described above.

Fig. 3. Student dashboard — enrolled clubs, registered events, and role-specific navigation.

Fig. 4. Club Core Lead dashboard — pending membership request awaiting approval.

Fig. 5. Public club discovery page — club cards with category filters and membership status.

Fig. 6. Admin dashboard — college-wide club registry, roles, and faculty coordinator management.

VI. Comparison with Existing Systems

Feature	Manual / Forms	Generic College System	UniSphere
Centralized club discovery	✗ (scattered links/groups)	△ (varies by system)	✓ dedicated club directory
Structured membership requests with approval	✗ (paper/duplicate forms)	△ (basic forms, if present)	✓ role-based request/approval flow
Event registration with seat tracking	✗ (manual sign-up sheets)	△ (not always supported)	✓ integrated seat tracking
Event photo albums (view/download)	✗ (shared informally)	△ (rarely a dedicated feature)	✓ per-event albums
Role-based dashboards	✗ (none)	△ (often one generic view)	✓ tailored per role
Centralized announcements	✗ (chat groups)	△ (sometimes present)	✓ built into the platform

Table III. Feature comparison against manual workflows and generic college management approaches.

This comparison is based on how manual, form-based club coordination is typically observed to work, alongside the general feature set of common college management platforms; it is not based on a formal audit of any specific competing product. The main difference UniSphere offers is that club discovery, membership approval, event registration, and event photo management are built in as core, connected features rather than being handled through separate tools or left out altogether.

VII. Conclusion

UniSphere brings club discovery, membership requests, event registration, announcements, and event photo galleries into one system, organised around a five-tier role hierarchy — Visitor, Student, Club Core Lead, Faculty Coordinator, and Admin — so that each type of user sees only the information and actions relevant to them. This addresses the specific problems described in Section I: scattered announcements, duplicate or unverifiable membership records, manual event sign-ups with no seat tracking, and event photos that are hard to find after the fact. Manual testing of the core workflows suggests the current implementation behaves as intended for these use cases, though further formal testing would be needed before wider deployment.

VIII. Future Scope

Several extensions could build on the current implementation. Adopting a more scalable, structured database as the number of clubs, events, and users grows would help the system handle larger campuses or multiple institutions. QR or barcode-based ticket scanning, following standards such as ISO/IEC 18004 [5], could replace manual seat tracking at the door with a quick scan-based check-in. Digital certificates with a publicly verifiable ID could be added so that a recruiter or another institution could confirm a student's participation without contacting the club directly. Tracking volunteer and co-curricular hours in a structured way would also help institutions compile the kind of accreditation data bodies such as NAAC and NBA ask for, which is currently difficult to gather from manual club records. Finally, making the system easier to access across devices connected to the same campus network would help multiple coordinators manage an event together without additional configuration.

References

[1] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.

[2] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," ACM Transactions on Information and System Security, vol. 4, no. 3, pp. 224–274, 2001.

[3] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015.

[4] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[5] ISO/IEC 18004:2015, Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification, International Organization for Standardization, 2015.